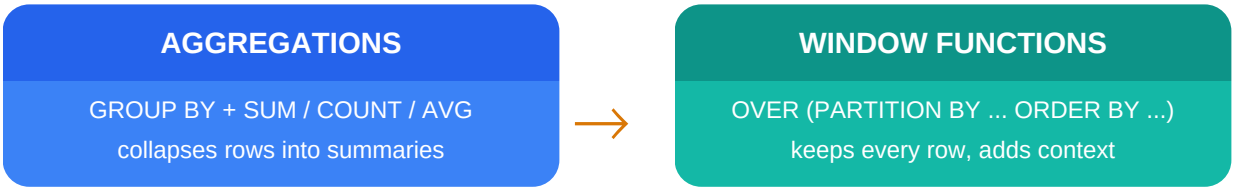
From Aggregations to Window Functions

A visual SQL guide built on the **gold_dim_customers** table



This guide walks through both techniques side by side, using the same customer dimension table so you can see exactly what changes when you move from a GROUP BY summary to a window function that keeps row-level detail.

Table used in every example

gold_dim_customers — customer_key, customer_id, customer_number, first_name, last_name, country, marital_status, gender, birthdate, create_date

1 Aggregations

Aggregations use **GROUP BY** together with functions like COUNT, SUM, AVG, MIN, and MAX to collapse many rows into one summary row per group. You trade row-level detail for a compact answer.

Example: customers per country

```
SELECT
  country,
  COUNT(*) AS total_customers
FROM gold_dim_customers
GROUP BY country
ORDER BY total_customers DESC;
```

country	total_customers
Australia	3,591
United States	7,482
United Kingdom	1,913
...	...

Sample values shown for illustration; run the query on your data for exact counts.

> **The limitation**
> Once grouped, individual customer rows (first_name, birthdate, etc.) are gone. You can no longer ask *which customer within each country ranks highest?* from this result — that detail was collapsed away.

Common aggregate functions

Function	Purpose
COUNT()	Number of rows / non-null values
SUM()	Total of a numeric column
AVG()	Mean of a numeric column
MIN() / MAX()	Smallest / largest value

2 Window Functions

A **window function** performs a calculation across a set of related rows (its window) using **OVER()**, but unlike GROUP BY, it does not collapse the rows. Every original row stays in the result, now enriched with the aggregate or ranking value calculated for its group.

Same question, row-level answer

```
SELECT
  customer_id,
  first_name,
  last_name,
  country,
  COUNT(*) OVER (PARTITION BY country) AS customers_in_country
FROM gold_dim_customers
ORDER BY country, customer_id;
```

customer_id	first_name	country	customers_in_country
11000	Jon	Australia	3,591
11001	Eugene	Australia	3,591
11002	Ruben	Australia	3,591
...	...	...	...

Why this matters

Every customer row survives. The window function just attaches a group-level fact (customers_in_country) onto each row, so it can be filtered, ranked, or compared against the customer's own record in the very same query.

Ranking customers within each country

```
SELECT
  customer_id,
  first_name,
  country,
  birthdate,
  ROW_NUMBER() OVER (
    PARTITION BY country
    ORDER BY birthdate ASC
  ) AS birth_order_in_country
FROM gold_dim_customers;
```

This assigns 1, 2, 3... to customers inside each country, ordered by birthdate — something a plain GROUP BY simply cannot produce, since it can only return one row per country.

3 Side-by-Side Comparison

Aspect	Aggregation (GROUP BY)	Window Function (OVER)
Row count in output	One row per group	One row per original row
Detail columns	Lost unless also grouped	Fully preserved
Keyword	GROUP BY	OVER (PARTITION BY ... ORDER BY ...)
Can rank rows?	No	Yes: ROW_NUMBER, RANK, DENSE_RANK
Can compare to prior/next row?	No	Yes: LAG, LEAD
Can show running totals?	No	Yes: SUM() OVER (ORDER BY ...)
Filters after	HAVING	Needs a subquery / CTE (WHERE can't see the window co

Window function cheat sheet

Function	What it returns
`ROW_NUMBER()`	Unique sequential number per row in the window
`RANK()`	Rank with gaps after ties (1,1,3...)
`DENSE_RANK()`	Rank without gaps after ties (1,1,2...)
`SUM()/AVG()/COUNT() OVER(...)`	Running or group total alongside each row
`LAG(col, n)`	Value from n rows before the current row
`LEAD(col, n)`	Value from n rows after the current row
`FIRST_VALUE() / LAST_VALUE()`	First / last value in the window frame

> **Takeaway**
>
> Rule of thumb: reach for **GROUP BY** when you want a summary table with fewer rows than you started with. Reach for a **window function** when you want to keep every row but still compare it against a group, a running total, or a neighboring row.